

Вопрос 1

Ответ сохранен

Балл: 1,00

Какие методы используются для управления хэш-таблицей в unordered_map?

Выберите один или несколько ответов:

- ☐ a. sort() — сортировка бакетов
- ☒ b. max_load_factor() — установка коэффициента загрузки
- ☒ c. reserve() — резервирование бакетов под N элементов
- ☐ d. merge() — слияние хэш-таблиц
- ☒ e. rehash() — принудительное изменение количества бакетов

Вопрос 2

Ответ сохранен

Балл: 1,00

Какие утверждения о `std::unordered_map` верны?

Выберите один или несколько ответов:

- ☐ a. Реализован как красно-черное дерево
- ☒ b. Элементы не упорядочены
- ☐ c. Элементы отсортированы по ключу
- ☒ d. Требует `std::hash` и `operator==`
- ☒ e. Средняя сложность операций $O(1)$
- ☒ f. Реализован как хэш-таблица

Вопрос 3

Ответ сохранен

Балл: 1,00

Какие категории итераторов существуют в C++?

Выберите один или несколько ответов:

- ☒ a. OutputIterator (запись, однонаправленный)
- ☒ b. BidirectionalIterator (двунаправленный: list, map, set)
- ☐ c. StreamIterator
- ☒ d. ForwardIterator (чтение/запись, однонаправленный)
- ☒ e. InputIterator (чтение, однонаправленный)
- ☐ f. PointerIterator
- ☒ g. RandomAccessIterator (произвольный доступ: vector, deque, array)

Вопрос 4

Пока нет ответа

Балл: 1,00

Какие утверждения о `std::map` верны?

Выберите один или несколько ответов:

- ☐ a. Реализован как хэш-таблица
- ☒ b. Реализован как красно-черное дерево
- ☒ c. Ключи уникальны
- ☒ d. Поиск/вставка/удаление: $O(\log n)$
- ☐ e. Поиск/вставка/удаление: $O(1)$
- ☒ f. Элементы отсортированы по ключу

Вопрос 5

Частично правильный

Баллов: 0,67 из 1,00

Что такое бакеты (buckets) в `std::unordered_map`?

Выберите один или несколько ответов:

☒ а. Корзины (ведра), куда складываются элементы с одинаковым хэш-значением 

☐ б. Непрерывная область памяти для всех элементов

☐ в. Коэффициент загрузки = $\text{size} / \text{bucket_count}$

☐ г. Отсортированный массив элементов

☒ е. Каждый бакет — связный список (цепочка коллизий) 



Ответ частично правильный.

Правильные ответы: Корзины (ведра), куда складываются элементы с одинаковым хэш-значением, Каждый бакет — связный список (цепочка коллизий), Коэффициент загрузки = $\text{size} / \text{bucket_count}$

Вопрос 6

Пока нет ответа

Балл: 1,00

Какие методы используются для работы с `std::map`?

Выберите один или несколько ответов:

- ☒ a. `emplace()` — создание элемента на месте
- ☒ b. `find()` — поиск по ключу, возвращает итератор
- ☒ c. `operator[]` — доступ/вставка по ключу
- ☐ d. `pop_front()`
- ☒ e. `insert()` — вставка пары ключ-значение
- ☐ f. `push_back()`

Вопрос 7

Частично правильный

Баллов: 0,25 из 1,00

Какие адаптеры итераторов предоставляет STL?

Выберите один или несколько ответов:

☒ a. `insert_iterator` (insert в позицию) ☒

☒ b. `front_insert_iterator` (push_front) ☒

☐ c. `const_iterator`

☒ d. `move_iterator` ☒

☒ e. `back_insert_iterator` (push_back) ☒

☐ f. `reverse_iterator` (обратный обход)



Ответ частично правильный.

Правильные ответы: `reverse_iterator` (обратный обход), `back_insert_iterator` (push_back), `front_insert_iterator` (push_front), `insert_iterator` (insert в позицию)

Вопрос 8

Пока нет ответа

Балл: 1,00

Какие категории итераторов соответствуют разным контейнерам?

Выберите один или несколько ответов:

- ☒ a. vector, deque, array — RandomAccessIterator
- ☒ b. list, map, set — BidirectionalIterator
- ☒ c. forward_list — ForwardIterator
- ☒ d. unordered_map, unordered_set — ForwardIterator
- ☐ e. Все контейнеры поддерживают RandomAccessIterator
- ☐ f. list поддерживает RandomAccessIterator

Вопрос 9

Пока нет ответа

Балл: 1,00

Для каких целей используются move итераторы (std::move_iterator)?

Выберите один или несколько ответов:

- ☒ a. Используются для эффективного перемещения элементов вместо копирования
- ☐ b. Для обратного обхода контейнера
- ☐ c. Для работы с файловыми потоками
- ☒ d. Преобразуют разыменованное значение в rvalue-ссылку

Вопрос 10

Пока нет ответа

Балл: 1,00

Какие stream итераторы предоставляет STL?

Выберите один или несколько ответов:

- ☐ a. `std::socket_iterator`
- ☐ b. `std::file_iterator`
- ☒ c. `std::istream_iterator` — чтение из потока ввода
- ☒ d. `std::ostream_iterator` — запись в поток вывода